Documento de apoyo (Sesión 6, Unidad 2)

Análisis y ecosistemas Big Data con Apache Spark

Fundamentos, arquitectura, exploración y carga desde data lakes

Javier Andrés Valdez González / Auxiliar

Seminario de Sistemas 2 — Sección P

Universidad de San Carlos de Guatemala

Marzo de 2026

Resumen

El análisis Big Data requiere arquitecturas distribuidas capaces de procesar grandes volúmenes de información con velocidad y flexibilidad. En esta sesión se introducen los fundamentos conceptuales de Big Data a través del marco de las 4V, y se desarrolla el papel de Apache Spark como motor de análisis distribuido para cargas batch e interactivas. El documento presenta la arquitectura de Spark, sus componentes principales, el flujo básico de trabajo para análisis de datos y prácticas de exploración con DataFrames. Finalmente, se aborda la carga de información desde data lakes, destacando formatos columnares y consideraciones de particionamiento para mejorar desempeño analítico.

Palabras clave: Big Data, 4V, Apache Spark, DataFrames, data lake, particionamiento, Parquet.

Índice

1. Introducción **3**

2. Análisis y ecosistemas Big Data: ¿Qué es Big Data? (4V) **4**

3. Análisis y ecosistemas Big Data: Arquitectura de Spark **5**

4. Análisis y ecosistemas Big Data: Fundamentos de Spark para análisis **6**

5. Análisis y ecosistemas Big Data: Exploración de datos en Spark **7**

6. Análisis y ecosistemas Big Data: Carga desde data lakes **8**

6.1. Formatos comunes de lectura 8

6.2. Ejemplo de carga desde data lake 8

7. Buenas prácticas mínimas para la semana 6 **9**

8. Conclusiones **10**

1. Introducción

Las organizaciones modernas generan datos desde aplicaciones transaccionales, sensores, plataformas web, redes sociales y sistemas empresariales. Este crecimiento no solo incrementa el volumen de datos, sino también la diversidad de formatos y la velocidad con la que la información cambia. En ese contexto surge Big Data como un enfoque para almacenar, procesar y analizar datos que superan las capacidades de herramientas tradicionales [5].

Apache Spark se ha consolidado como uno de los motores más utilizados para análisis distribuido por su modelo de ejecución en memoria, su API unificada y su integración con ecosistemas de almacenamiento masivo. Spark permite construir pipelines de transformación, exploración y análisis sobre datos estructurados y semiestructurados en entornos locales, clústeres on-premise o nube [2].

El objetivo de este documento es presentar, en formato técnico y práctico, los temas de la semana 6: definición de Big Data (4V), arquitectura de Spark, fundamentos de análisis en Spark, exploración de datos y carga desde data lakes.

2. Análisis y ecosistemas Big Data: ¿Qué es Big Data? (4V)

Big Data describe conjuntos de datos cuya escala, complejidad y dinámica requieren arquitecturas distribuidas y técnicas especializadas para su procesamiento [6]. Una forma estándar de entenderlo es mediante las 4V:

V1. Volumen: cantidad masiva de datos (GB, TB, PB) provenientes de múltiples fuentes.

V2. Velocidad: ritmo al que los datos se generan, transmiten y deben procesarse.

V3. Variedad: coexistencia de formatos estructurados, semiestructurados y no estructurados.

V4. Veracidad: nivel de calidad, consistencia y confiabilidad del dato para análisis.

Estas dimensiones obligan a diseñar soluciones que combinen almacenamiento escalable, procesamiento paralelo, control de calidad y gobierno de datos.

3. Análisis y ecosistemas Big Data: Arquitectura de Spark

Spark opera sobre un modelo distribuido orientado a clúster, donde una aplicación se divide en tareas paralelas ejecutadas por nodos trabajadores. Sus componentes principales son:

- **Driver:** proceso que contiene la lógica de la aplicación, planifica tareas y coordina la ejecución.
- **Cluster Manager:** asigna recursos entre aplicaciones (por ejemplo, Standalone, YARN o Kubernetes).
- **Executors:** procesos en nodos trabajadores que ejecutan tareas y almacenan datos en memoria/disco.
- **SparkSession:** punto de entrada para trabajar con DataFrames, SQL y lectura/escritura de datos.

A nivel lógico, Spark transforma el código en un DAG (grafo acíclico dirigido) de etapas y tareas. Este plan se optimiza antes de ejecutarse para reducir movimientos de datos y mejorar rendimiento [3].

4. Análisis y ecosistemas Big Data: Fundamentos de Spark para análisis

En análisis de datos, Spark se usa principalmente mediante DataFrames y Spark SQL. El flujo básico de trabajo suele ser:

F1. Inicializar la sesión de Spark.

F2. Cargar datos desde archivos o tablas.

F3. Transformar (filtrar, agregar, limpiar, enriquecer).

F4. Analizar con operaciones SQL o API de DataFrame.

F5. Persistir resultados en formatos analíticos.

Listing 1: Ejemplo base de análisis con PySpark DataFrame

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum as sum_

spark = SparkSession.builder \
    .appName("Semana6-BigData") \
    .getOrCreate()

df = spark.read.option("header", True).csv("data/ventas.csv")

resumen = (
    df.groupBy("categoria")
      .agg(sum_(col("monto").cast("double")).alias("ventas_totales")
         )
      .orderBy(col("ventas_totales").desc())
)

resumen.show(20, truncate=False)
```

5. Análisis y ecosistemas Big Data: Exploración de datos en Spark

La exploración inicial permite entender estructura, calidad y patrones antes de modelar. En Spark, algunas operaciones esenciales son:

- `printSchema()` para validar tipos de datos.
- `show()` para inspección rápida de registros.
- `describe()` para estadísticas descriptivas básicas.
- `count()`, `distinct()` y `groupBy()` para perfiles de distribución.
- análisis de nulos y valores atípicos con reglas de negocio.

Listing 2: Rutina básica de exploración en Spark

```
from pyspark.sql.functions import col, count, when

print("Total de filas:", df.count())

df.printSchema()
df.show(10, truncate=False)

df.describe().show()

nulos = df.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in df.columns
])

nulos.show(truncate=False)
```

Una exploración adecuada evita errores posteriores de agregación, sesgo por datos faltantes y fallos de tipado en cálculos.

6. Análisis y ecosistemas Big Data: Carga desde data lakes

Un data lake almacena datos en bruto y curados, generalmente en almacenamiento de objetos o sistemas distribuidos, para su consumo analítico por distintos motores [1]. Spark se integra de forma natural con esta arquitectura gracias a sus conectores y soporte de formatos abiertos.

6.1. Formatos comunes de lectura

- **CSV/JSON:** útiles para ingestión inicial, pero menos eficientes para análisis masivo.
- **Parquet/ORC:** formatos columnares comprimidos y optimizados para consultas analíticas.
- **Delta Lake (cuando aplica):** agrega transaccionalidad y versionado sobre archivos de data lake [4].

6.2. Ejemplo de carga desde data lake

Listing 3: Lectura y escritura en formato Parquet particionado

```
from pyspark.sql.functions import year, month, to_date, col

raw = spark.read.parquet("s3a://mi-datalake/raw/ventas/")

curated = (
    raw.withColumn("fecha", to_date(col("fecha")))
       .withColumn("anio", year(col("fecha")))
       .withColumn("mes", month(col("fecha")))
       .filter(col("monto").isNotNull())
)

(curated.write
    .mode("overwrite")
    .partitionBy("anio", "mes")
    .parquet("s3a://mi-datalake/curated/ventas/"))
```

El particionamiento por fecha facilita el *pruning* de particiones, reduciendo el volumen leído y acelerando consultas.

7. Buenas prácticas mínimas para la semana 6

Tema	**Práctica recomendada**
Lectura de datos	Definir esquema explícito cuando sea posible para evitar inferencia costosa o errores de tipos.
Transformaciones	Encadenar operaciones de forma declarativa y validar resultados parciales con muestras controladas.
Exploración	Revisar nulos, cardinalidad y rangos antes de construir agregaciones o métricas de negocio.
Data lake	Preferir Parquet para capas analíticas y organizar rutas particionadas por columnas de alta selectividad.
Rendimiento	Evitar acciones innecesarias repetidas; usar cache solo cuando un DataFrame se reutiliza varias veces.

8. Conclusiones

Big Data se entiende mejor como un problema de ingeniería de datos definido por volumen, velocidad, variedad y veracidad. En este contexto, Spark aporta un modelo distribuido y unificado para procesar datos a escala con APIs orientadas a análisis.

Para la práctica de esta semana, los puntos críticos son: comprender la arquitectura de Spark, dominar el flujo base con DataFrames, explorar datos con criterios de calidad y cargar/organizar información desde data lakes con formatos eficientes. Estos fundamentos son la base para construir pipelines analíticos robustos en sesiones posteriores.

Referencias

[1] Amazon Web Services. «What is a data lake?» Visitado 8 de mar. de 2026. dirección: https://aws.amazon.com/what-is/data-lake/

[2] Apache Spark Project. «Apache Spark - Overview,» visitado 8 de mar. de 2026. dirección: https://spark.apache.org/docs/latest/

[3] Apache Spark Project. «Spark SQL, DataFrames and Datasets Guide,» visitado 8 de mar. de 2026. dirección: https://spark.apache.org/docs/latest/sql-programming-guide.html

[4] Delta Lake Project. «Delta Lake Documentation,» visitado 8 de mar. de 2026. dirección: https://docs.delta.io/latest/index.html

[5] IBM. «What is Big Data?» Visitado 8 de mar. de 2026. dirección: https://www.ibm.com/think/topics/big-data

[6] D. Laney, «3D Data Management: Controlling Data Volume, Velocity, and Variety,» en *META Group Research Note*, 2001.